Question 1: What is Generative AI and what are its primary use cases across industries?

Generative AI is a type of artificial intelligence that can **create new content such as text, images, audio, video, or code** by learning patterns from large datasets. It typically uses advanced deep learning models such as **transformers, generative adversarial networks (GANs), and large language models** to generate realistic and contextually relevant outputs.

Primary Use Cases Across Industries

1. **Content Creation and Marketing** Generative AI helps create articles, social media posts, product descriptions, and advertisements, improving productivity in media and digital marketing.

2. **Healthcare and Drug Discovery** It assists in generating molecular structures, analyzing medical data, and supporting clinical research to accelerate drug development.

3. **Customer Support and Virtual Assistants** AI-powered chatbots and conversational systems automatically respond to customer queries, improving service efficiency.

4. **Software Development** Generative AI can generate code snippets, automate debugging, and assist developers in building applications faster.

5. **Design and Creative Industries** It enables the creation of images, music, videos, and graphic designs, supporting creative professionals.

Summary

Generative AI is transforming industries by **automating creative tasks, enhancing productivity, and enabling intelligent content generation** across domains such as business, healthcare, technology, and media.

Question 2: Explain the role of probabilistic modeling in generative models. How do these models differ from discriminative models?

Probabilistic modeling plays a central role in generative models by representing the **probability distribution of data**. These models learn how data is generated by estimating the **joint probability distribution (P(X, Y))**, where (X) represents the input data and (Y) represents labels or outcomes. By learning this distribution, generative models can **create new samples that**

resemble the training data, making them useful for tasks such as text generation, image synthesis, and data augmentation.

Role of Probabilistic Modeling

1. **Data Generation** – Models learn the underlying distribution of the data and generate new realistic samples.
2. **Handling Uncertainty** – Probabilistic approaches allow models to represent multiple possible outcomes.
3. **Missing Data Estimation** – They can infer or reconstruct missing parts of data.
4. **Flexible Learning** – Useful for unsupervised and semi-supervised learning.

Difference Between Generative and Discriminative Models

Aspect	Generative Models	Discriminative Models
Probability Learned	Joint probability (P(X, Y))	Conditional probability (P(Y X))
Goal	Model how data is generated	Predict labels from data
Capability	Can generate new data samples	Cannot generate new data
Examples	Naive Bayes, GANs, Variational Autoencoders	Logistic Regression, SVM, Neural Networks

Summary

Generative models use probabilistic modeling to **understand and reproduce the underlying data distribution**, while discriminative models focus only on **distinguishing between classes for prediction tasks**.

Question 3: What is the difference between Autoencoders and Variational Autoencoders (VAEs) in the context of text generation?

Autoencoders (AE) and **Variational Autoencoders (VAE)** are neural network models used for learning compact representations of data, but they differ in how they encode information and generate new data.

Autoencoders (AE)

An autoencoder consists of two main parts: an **encoder** and a **decoder**.

- The **encoder** compresses the input data into a fixed-size latent representation (vector).
- The **decoder** reconstructs the original data from this latent vector.

In text generation, autoencoders learn to reproduce the input sentence, but they do **not explicitly model the probability distribution of the latent space**, making it difficult to generate diverse or meaningful new sentences.

Variational Autoencoders (VAE)

A VAE is a probabilistic extension of the autoencoder. Instead of encoding data into a single point in the latent space, it encodes it into a **probability distribution (mean and variance)**. New samples are generated by **sampling from this distribution** and decoding them into text.

Key Differences

Aspect	Autoencoder	Variational Autoencoder
Latent Representation	Fixed vector	Probability distribution
Sampling	Not designed for sampling	Allows random sampling
Data Generation	Limited	Generates diverse outputs
Use in Text Generation	Mainly reconstruction	Effective for generating new text

Summary

Autoencoders focus on **reconstructing input data**, while VAEs introduce **probabilistic latent spaces**, enabling them to generate **new and varied text samples**, which makes them more suitable for generative NLP tasks.

Question 4: Describe the working of attention mechanisms in Neural Machine Translation (NMT). Why are they critical?

Attention mechanisms in Neural Machine Translation (NMT) allow the model to focus on the most relevant parts of the source sentence while generating each word in the translated sentence. Instead of relying on a single fixed representation of the input, attention dynamically assigns weights to different words in the input sequence.

Working of Attention in NMT

1. **Encoding the Source Sentence** The encoder processes the input sentence and produces a sequence of hidden states representing each word.

2. **Calculating Attention Scores** When the decoder generates a word, it computes attention scores between its current hidden state and each encoder hidden state.

3. **Generating Attention Weights** These scores are converted into normalized weights using a softmax function, indicating the importance of each input word.

4. **Creating a Context Vector** The weighted sum of encoder hidden states forms a context vector, which provides relevant information from the input sentence.
5. **Producing the Output Word** The decoder uses the context vector along with its hidden state to predict the next word in the translated sentence.

Why Attention is Critical

- **Improves Translation Quality** by focusing on relevant words in long sentences.
- **Handles Long Sequences** better than basic encoder–decoder models.
- **Captures Word Alignment** between source and target languages.
- **Reduces Information Loss** caused by compressing the entire input into a single vector.

Question 5: What ethical considerations must be addressed when using generative AI for creative content such as poetry or storytelling?

When using **generative AI for creative content such as poetry or storytelling**, several ethical considerations must be addressed to ensure responsible and fair use of the technology.

Copyright and Intellectual Property

Generative AI models are trained on large datasets that may include copyrighted material. It is important to ensure that generated content does not replicate or closely imitate protected works without proper permission.

Plagiarism and Originality

AI-generated content should maintain originality and avoid copying existing texts. Proper disclosure should be provided when creative works are generated with AI assistance.

Bias and Fairness

Training data may contain social or cultural biases, which can lead to biased or offensive outputs in generated stories or poems. Developers must monitor and reduce such biases.

Transparency and Disclosure

Audiences should be informed when content is generated or assisted by AI to maintain transparency and trust.

Misuse and Misinformation

Generative AI could be misused to create misleading or harmful narratives. Ethical guidelines and moderation mechanisms should be implemented to prevent misuse.

Addressing these considerations helps ensure that generative AI is used **responsibly, transparently, and ethically in creative fields**.

Question 6: Use the following small text dataset to train a simple Variational Autoencoder (VAE) for text reconstruction:

["The sky is blue", "The sun is bright", "The grass is green", "The night is dark", "The stars are shining"]

1. Preprocess the data (tokenize and pad the sequences).
2. Build a basic VAE model for text reconstruction.
3. Train the model and show how it reconstructs or generates similar sentences. Include your code, explanation, and sample outputs.

```
# 1. Data Preprocessing (Tokenization and Padding)

import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

texts = [
"The sky is blue",
"The sun is bright",
"The grass is green",
"The night is dark",
"The stars are shining"
]

tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)
max_len = max(len(seq) for seq in sequences)

X = pad_sequences(sequences, maxlen=max_len, padding='post')

vocab_size = len(tokenizer.word_index) + 1

print(X)
```

```
[[ 1  3  2  4]
 [ 1  5  2  6]
 [ 1  7  2  8]
```

```
 [ 1  9  2 10]
 [ 1 11 12 13]]
```

```python
# 2. Build a Basic Variational Autoencoder (VAE)

import tensorflow as tf
from tensorflow.keras import layers, ops, Model, Sequential

latent_dim = 2
input_dim = max_len

class VAE(Model):
    def __init__(self, input_dim, latent_dim, **kwargs):
        super().__init__(**kwargs)
        self.input_dim = input_dim
        self.latent_dim = latent_dim

        # Encoder Model
        encoder_inputs = layers.Input(shape=(input_dim,))
        x = layers.Dense(16, activation='relu')(encoder_inputs)
        z_mean = layers.Dense(latent_dim)(x)
        z_log_var = layers.Dense(latent_dim)(x)
        self.encoder = Model(encoder_inputs, [z_mean, z_log_var],
name="encoder")

        # Decoder Model
        decoder_inputs = layers.Input(shape=(latent_dim,))
        x = layers.Dense(16, activation='relu')(decoder_inputs)
        decoder_outputs = layers.Dense(input_dim)(x)
        self.decoder = Model(decoder_inputs, decoder_outputs,
name="decoder")

    def encode(self, data):
        z_mean, z_log_var = self.encoder(data)
        epsilon = tf.random.normal(shape=(tf.shape(z_mean)[0],
self.latent_dim))
        z = z_mean + tf.exp(0.5 * z_log_var) * epsilon
        return z_mean, z_log_var, z

    def train_step(self, data):
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encode(data)
            reconstruction = self.decoder(z)
            reconstruction_loss = ops.mean(ops.square(data -
reconstruction))
            kl_loss = -0.5 * ops.mean(1 + z_log_var -
ops.square(z_mean) - ops.exp(z_log_var))
            total_loss = reconstruction_loss + kl_loss

        grads = tape.gradient(total_loss, self.trainable_weights)
```

```
        self.optimizer.apply_gradients(zip(grads,
self.trainable_weights))
        return {"loss": total_loss, "reconstruction_loss":
reconstruction_loss, "kl_loss": kl_loss}

    def call(self, inputs):
        z_mean, z_log_var, z = self.encode(inputs)
        return self.decoder(z)

vae = VAE(input_dim, latent_dim)
vae.compile(optimizer='adam')

# Build the model weights
vae(tf.zeros((1, input_dim)))
vae.summary()
```

```
Model: "vae_1"

┏━━━
```

```
51.4661 - reconstruction_loss: 38.6660
Epoch 4/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - kl_loss: 16.3527 - loss:
132.0291 - reconstruction_loss: 115.6764
Epoch 5/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 3.3120 - loss:
20.1032 - reconstruction_loss: 16.7912
Epoch 6/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 2.9980 - loss:
24.6598 - reconstruction_loss: 21.6618
Epoch 7/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 21.2144 - loss:
119.0090 - reconstruction_loss: 97.7946
Epoch 8/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 0.7774 - loss:
10.8279 - reconstruction_loss: 10.0505
Epoch 9/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 10.4695 - loss:
155.1039 - reconstruction_loss: 144.6344
Epoch 10/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 2.0268 - loss:
23.8616 - reconstruction_loss: 21.8348
Epoch 11/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 12.0777 - loss:
59.1846 - reconstruction_loss: 47.1069
Epoch 12/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 1.6990 - loss:
19.8345 - reconstruction_loss: 18.1355
Epoch 13/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 0.5206 - loss:
10.1871 - reconstruction_loss: 9.6665
Epoch 14/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 1.4534 - loss:
18.8454 - reconstruction_loss: 17.3920
Epoch 15/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 0.4545 - loss: 7.1866
- reconstruction_loss: 6.7321
Epoch 16/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - kl_loss: 7.5994 - loss:
118.8011 - reconstruction_loss: 111.2017
Epoch 17/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - kl_loss: 1.2005 - loss:
17.6824 - reconstruction_loss: 16.4819
Epoch 18/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 6.7272 - loss:
59.7413 - reconstruction_loss: 53.0142
Epoch 19/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 0.3428 - loss: 7.0505
- reconstruction_loss: 6.7077
```

```
Epoch 20/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - kl_loss: 5.9912 - loss:
57.0026 - reconstruction_loss: 51.0115
Epoch 21/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 6.2613 - loss:
122.1921 - reconstruction_loss: 115.9308
Epoch 22/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 6.1298 - loss:
119.9582 - reconstruction_loss: 113.8284
Epoch 23/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - kl_loss: 4.9539 - loss:
46.8595 - reconstruction_loss: 41.9056
Epoch 24/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - kl_loss: 5.8835 - loss:
101.7959 - reconstruction_loss: 95.9125
Epoch 25/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 0.2418 - loss: 6.6835
- reconstruction_loss: 6.4416
Epoch 26/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - kl_loss: 4.2764 - loss:
43.9134 - reconstruction_loss: 39.6370
Epoch 27/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 5.4992 - loss:
99.2070 - reconstruction_loss: 93.7077
Epoch 28/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 5.3986 - loss:
100.2073 - reconstruction_loss: 94.8087
Epoch 29/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 0.2042 - loss: 6.5106
- reconstruction_loss: 6.3064
Epoch 30/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 3.7396 - loss:
44.3720 - reconstruction_loss: 40.6324
Epoch 31/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 5.2378 - loss:
101.6878 - reconstruction_loss: 96.4501
Epoch 32/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - kl_loss: 0.1987 - loss: 6.2162
- reconstruction_loss: 6.0176
Epoch 33/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - kl_loss: 3.5343 - loss:
48.4863 - reconstruction_loss: 44.9520
Epoch 34/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - kl_loss: 0.2088 - loss: 5.8376
- reconstruction_loss: 5.6288
Epoch 35/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - kl_loss: 3.4138 - loss:
43.4058 - reconstruction_loss: 39.9920
Epoch 36/100
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - kl_loss: 0.2249 - loss: 6.9925
- reconstruction_loss: 6.7676
Epoch 37/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - kl_loss: 5.8967 - loss:
97.3579 - reconstruction_loss: 91.4612
Epoch 38/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - kl_loss: 0.2443 - loss: 6.2961
- reconstruction_loss: 6.0518
Epoch 39/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 0.6677 - loss:
13.7682 - reconstruction_loss: 13.1005
Epoch 40/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - kl_loss: 0.2613 - loss: 5.4475
- reconstruction_loss: 5.1863
Epoch 41/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - kl_loss: 1.5310 - loss:
26.0772 - reconstruction_loss: 24.5461
Epoch 42/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 1.5371 - loss:
23.3006 - reconstruction_loss: 21.7635
Epoch 43/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - kl_loss: 0.7040 - loss:
12.9269 - reconstruction_loss: 12.2229
Epoch 44/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 1.5467 - loss:
22.8448 - reconstruction_loss: 21.2981
Epoch 45/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 1.5625 - loss:
22.5592 - reconstruction_loss: 20.9967
Epoch 46/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 3.0820 - loss:
53.5043 - reconstruction_loss: 50.4223
Epoch 47/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 0.7649 - loss:
13.1328 - reconstruction_loss: 12.3679
Epoch 48/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 7.9577 - loss:
92.3214 - reconstruction_loss: 84.3637
Epoch 49/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - kl_loss: 0.8261 - loss:
11.4909 - reconstruction_loss: 10.6648
Epoch 50/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - kl_loss: 0.3817 - loss: 5.4664
- reconstruction_loss: 5.0847
Epoch 51/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - kl_loss: 0.8984 - loss:
11.5087 - reconstruction_loss: 10.6103
Epoch 52/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - kl_loss: 1.8101 - loss:
```

```
20.7606 - reconstruction_loss: 18.9506
Epoch 53/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - kl_loss: 0.4369 - loss: 5.5049
- reconstruction_loss: 5.0680
Epoch 54/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - kl_loss: 3.3290 - loss:
42.8915 - reconstruction_loss: 39.5625
Epoch 55/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - kl_loss: 10.1588 - loss:
77.9547 - reconstruction_loss: 67.7958
Epoch 56/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - kl_loss: 3.4959 - loss:
34.6427 - reconstruction_loss: 31.1468
Epoch 57/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - kl_loss: 0.5160 - loss: 5.0638
- reconstruction_loss: 4.5478
Epoch 58/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - kl_loss: 1.1448 - loss:
11.9246 - reconstruction_loss: 10.7798
Epoch 59/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 0.5544 - loss: 4.4085
- reconstruction_loss: 3.8541
Epoch 60/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - kl_loss: 2.2442 - loss:
21.3729 - reconstruction_loss: 19.1287
Epoch 61/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 3.8613 - loss:
47.3481 - reconstruction_loss: 43.4867
Epoch 62/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 2.3721 - loss:
27.3320 - reconstruction_loss: 24.9599
Epoch 63/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - kl_loss: 4.0438 - loss:
40.5987 - reconstruction_loss: 36.5548
Epoch 64/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 13.0222 - loss:
69.7390 - reconstruction_loss: 56.7168
Epoch 65/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 4.2710 - loss:
41.9963 - reconstruction_loss: 37.7253
Epoch 66/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 1.5185 - loss: 9.9170
- reconstruction_loss: 8.3985
Epoch 67/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 0.7564 - loss: 3.9721
- reconstruction_loss: 3.2157
Epoch 68/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 1.6167 - loss:
12.4566 - reconstruction_loss: 10.8399
```

```
Epoch 69/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - kl_loss: 0.8033 - loss: 5.4961
- reconstruction_loss: 4.6928
Epoch 70/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 15.2267 - loss:
58.0664 - reconstruction_loss: 42.8397
Epoch 71/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 4.8061 - loss:
30.1339 - reconstruction_loss: 25.3278
Epoch 72/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 0.8350 - loss: 3.5523
- reconstruction_loss: 2.7173
Epoch 73/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 3.0317 - loss:
17.7663 - reconstruction_loss: 14.7346
Epoch 74/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 15.5171 - loss:
58.3574 - reconstruction_loss: 42.8403
Epoch 75/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 1.7702 - loss: 8.3323
- reconstruction_loss: 6.5621
Epoch 76/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - kl_loss: 4.9150 - loss:
27.7720 - reconstruction_loss: 22.8570
Epoch 77/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 4.9114 - loss:
30.6239 - reconstruction_loss: 25.7125
Epoch 78/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 0.8803 - loss: 3.4921
- reconstruction_loss: 2.6118
Epoch 79/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 1.8070 - loss:
10.2058 - reconstruction_loss: 8.3988
Epoch 80/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 15.9790 - loss:
50.9829 - reconstruction_loss: 35.0039
Epoch 81/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 0.8978 - loss: 3.5386
- reconstruction_loss: 2.6408
Epoch 82/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 5.0203 - loss:
29.0559 - reconstruction_loss: 24.0355
Epoch 83/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 3.2509 - loss:
14.9252 - reconstruction_loss: 11.6744
Epoch 84/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 1.8985 - loss: 7.6110
- reconstruction_loss: 5.7125
Epoch 85/100
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 0.9369 - loss: 3.6219
- reconstruction_loss: 2.6850
Epoch 86/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 16.4929 - loss:
44.3820 - reconstruction_loss: 27.8891
Epoch 87/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 16.3886 - loss:
48.3894 - reconstruction_loss: 32.0008
Epoch 88/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 5.1466 - loss:
28.1364 - reconstruction_loss: 22.9898
Epoch 89/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - kl_loss: 16.3865 - loss:
48.3651 - reconstruction_loss: 31.9786
Epoch 90/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 1.9535 - loss: 7.0080
- reconstruction_loss: 5.0545
Epoch 91/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 3.4147 - loss:
16.0809 - reconstruction_loss: 12.6662
Epoch 92/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - kl_loss: 5.4023 - loss:
26.5040 - reconstruction_loss: 21.1017
Epoch 93/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 2.0451 - loss: 6.8786
- reconstruction_loss: 4.8335
Epoch 94/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - kl_loss: 5.5300 - loss:
22.8664 - reconstruction_loss: 17.3365
Epoch 95/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - kl_loss: 2.0796 - loss: 6.6831
- reconstruction_loss: 4.6035
Epoch 96/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - kl_loss: 17.1392 - loss:
45.0985 - reconstruction_loss: 27.9593
Epoch 97/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - kl_loss: 17.1072 - loss:
40.6265 - reconstruction_loss: 23.5193
Epoch 98/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - kl_loss: 5.4773 - loss:
22.8257 - reconstruction_loss: 17.3484
Epoch 99/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - kl_loss: 3.4907 - loss:
16.8851 - reconstruction_loss: 13.3944
Epoch 100/100
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - kl_loss: 16.7298 - loss:
39.3005 - reconstruction_loss: 22.5706

<keras.src.callbacks.history.History at 0x7828d44f7980>
```

```python
# 4. Reconstruct Sentences
X_float = tf.cast(X, tf.float32)
pred = vae.predict(X_float)

print("Original Token IDs:")
print(X)

print("\nReconstructed Token IDs (approximate):")
# We use round to see which word index the model is getting closest to
print(np.round(pred).astype(int))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 97ms/step
Original Token IDs:
[[ 1  3  2  4]
 [ 1  5  2  6]
 [ 1  7  2  8]
 [ 1  9  2 10]
 [ 1 11 12 13]]

Reconstructed Token IDs (approximate):
[[ 1  2  2  1]
 [ 1  5  4  2]
 [ 1  5  4  3]
 [ 1  7  5  3]
 [ 1 10  8  4]]
```

Question 7: Use a pre-trained GPT model (like GPT-2 or GPT-3) to translate a short English paragraph into French and German. Provide the original and translated text

```python
# ==============================================
# 2. Import Libraries
# ==============================================

from transformers import pipeline
import torch

# ==============================================
# 3. Load Pre-trained GPT-2 Model
# ==============================================

generator = pipeline("text-generation", model="gpt2")
```

```python
# ==============================================
# 4. Define English Paragraph
# ==============================================

english_text = """
Artificial Intelligence is transforming industries worldwide.
It improves efficiency, enables automation, and creates new
opportunities for innovation.
"""

print("========== ORIGINAL ENGLISH TEXT ==========\n")
print(english_text)

# ==============================================
# 5. Translate into French
# ==============================================

prompt_french = f"""
Translate the following English text into French:

{english_text}

French Translation:
"""

french_output = generator(prompt_french, max_length=200,
num_return_sequences=1)

# Extract only translated portion
french_translation = french_output[0]['generated_text'].split("French
Translation:")[-1].strip()

print("\n========== FRENCH TRANSLATION ==========\n")
print(french_translation)

# ==============================================
# 6. Translate into German
# ==============================================

prompt_german = f"""
Translate the following English text into German:

{english_text}

German Translation:
"""

german_output = generator(prompt_german, max_length=200,
num_return_sequences=1)

german_translation = german_output[0]['generated_text'].split("German
```

```
Translation:")[-1].strip()

print("\n========== GERMAN TRANSLATION ==========\n")
print(german_translation)
```

{"model_id":"e8cd402515e449acb053e47dd54c0904","version_major":2,"version_minor":0}

```
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                    | Status     |  |
-----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end
generation.
Both `max_new_tokens` (=256) and `max_length`(=200) seem to have been
set. `max_new_tokens` will take precedence. Please refer to the
documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

========== ORIGINAL ENGLISH TEXT ==========


Artificial Intelligence is transforming industries worldwide.
It improves efficiency, enables automation, and creates new
opportunities for innovation.


Setting `pad_token_id` to `eos_token_id`:50256 for open-end
generation.
Both `max_new_tokens` (=256) and `max_length`(=200) seem to have been
set. `max_new_tokens` will take precedence. Please refer to the
documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)


========== FRENCH TRANSLATION ==========

We are the inventors, engineers, leaders in an industry that has lost
the ability to communicate effectively.

It is an opportunity to make the world a better place,

and a place for the future.

To make the world a better place, it is up to us, the people. But to
```

do that, one must make bold, bold decisions. We will see the world transformed by this breakthrough technology.

France

For the first time, a French company is opening its doors to the private sector.

The company, called NAM, is an IT and telecom technology company, based in Paris, which is creating a new medium of communication.

Its team is already working on a new technology to "break" the "telephony world," using blockchain technology.

The company is set to launch its first digital bank, that will open in France in February 2016.

In September, the company will launch the first mobile app.

It will be used by 4.2 million people worldwide.

France's first digital bank

In France, the world's first mobile banking system is already under development. The company, a consortium of government, private and public sector entities, will open a

========== GERMAN TRANSLATION ==========

"In Germany, the machines are the new industrialists. They create new jobs, make new products, and create new industries.

Artificial Intelligence can do more than just create jobs. It can also create new jobs."

How does artificial intelligence work in our world?

According to a new study published in the International Journal of Human Development,

"The technology of artificial intelligence is in its third generation. It is the most advanced in the history of human civilization."

The work shows that artificial intelligence is able to "create, process, and share information, more efficiently, and with more accuracy."

The authors of this work

```
"In Germany, the machines are the new industrialists. They create new
jobs, make new products, and create new industries.

"The machines can do more than just
```

Question 8: Implement a simple attention-based encoder-decoder model for English-to-Spanish translation using Tensorflow or PyTorch.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# ==============================================
# 2. Small Toy Dataset
# ==============================================

english_sentences = [
    "hello",
    "how are you",
    "i am fine",
    "good morning",
    "thank you"
]

spanish_sentences = [
    "hola",
    "como estas",
    "estoy bien",
    "buenos dias",
    "gracias"
]

# Add start and end tokens
spanish_sentences = ["<start> " + s + " <end>" for s in
spanish_sentences]

# ==============================================
# 3. Tokenization
# ==============================================

eng_tokenizer = keras.preprocessing.text.Tokenizer(filters='')
```

```
eng_tokenizer.fit_on_texts(english_sentences)
eng_seq = eng_tokenizer.texts_to_sequences(english_sentences)
eng_max_len = max(len(seq) for seq in eng_seq)
eng_seq = keras.preprocessing.sequence.pad_sequences(eng_seq,
maxlen=eng_max_len, padding='post')

spa_tokenizer = keras.preprocessing.text.Tokenizer(filters='')
spa_tokenizer.fit_on_texts(spanish_sentences)
spa_seq = spa_tokenizer.texts_to_sequences(spanish_sentences)
spa_max_len = max(len(seq) for seq in spa_seq)
spa_seq = keras.preprocessing.sequence.pad_sequences(spa_seq,
maxlen=spa_max_len, padding='post')

eng_vocab_size = len(eng_tokenizer.word_index) + 1
spa_vocab_size = len(spa_tokenizer.word_index) + 1

print("English Vocabulary Size:", eng_vocab_size)
print("Spanish Vocabulary Size:", spa_vocab_size)

# ==============================================
# 4. Build Encoder
# ==============================================

embedding_dim = 64
units = 128

class Encoder(keras.Model):
    def __init__(self):
        super().__init__()
        self.embedding = layers.Embedding(eng_vocab_size,
embedding_dim)
        self.lstm = layers.LSTM(units, return_sequences=True,
return_state=True)

    def call(self, x):
        x = self.embedding(x)
        output, state_h, state_c = self.lstm(x)
        return output, state_h, state_c

# ==============================================
# 5. Bahdanau Attention
# ==============================================

class BahdanauAttention(layers.Layer):
    def __init__(self):
        super().__init__()
        self.W1 = layers.Dense(units)
        self.W2 = layers.Dense(units)
        self.V = layers.Dense(1)
```

```
    def call(self, encoder_outputs, hidden):
        hidden_with_time_axis = tf.expand_dims(hidden, 1)
        score = self.V(tf.nn.tanh(self.W1(encoder_outputs) +
self.W2(hidden_with_time_axis)))
        attention_weights = tf.nn.softmax(score, axis=1)
        context_vector = attention_weights * encoder_outputs
        context_vector = tf.reduce_sum(context_vector, axis=1)
        return context_vector, attention_weights



class Decoder(keras.Model):
    def __init__(self):
        super().__init__()
        self.embedding = layers.Embedding(spa_vocab_size,
embedding_dim)
        self.lstm = layers.LSTM(units, return_sequences=True,
return_state=True)
        self.fc = layers.Dense(spa_vocab_size)
        self.attention = BahdanauAttention()

    def call(self, x, hidden, cell, encoder_outputs):
        context_vector, attention_weights =
self.attention(encoder_outputs, hidden)
        x = self.embedding(x)
        x = tf.concat([tf.expand_dims(context_vector, 1), x], axis=-1)
        output, state_h, state_c = self.lstm(x, initial_state=[hidden,
cell])
        output = self.fc(output)
        return output, state_h, state_c


encoder = Encoder()
decoder = Decoder()

optimizer = keras.optimizers.Adam()
loss_object =
keras.losses.SparseCategoricalCrossentropy(from_logits=True)



epochs = 300

for epoch in range(epochs):
    total_loss = 0
    with tf.GradientTape() as tape:
        enc_output, enc_h, enc_c = encoder(eng_seq)

        dec_input = tf.expand_dims(spa_seq[:, 0], 1)
        dec_hidden, dec_cell = enc_h, enc_c
```

```
        for t in range(1, spa_max_len):
            predictions, dec_hidden, dec_cell = decoder(
                dec_input, dec_hidden, dec_cell, enc_output
            )
            loss = loss_object(spa_seq[:, t], predictions[:, 0])
            total_loss += loss
            dec_input = tf.expand_dims(spa_seq[:, t], 1)

    variables = encoder.trainable_variables +
decoder.trainable_variables
    gradients = tape.gradient(total_loss, variables)
    optimizer.apply_gradients(zip(gradients, variables))

    if epoch % 50 == 0:
        print(f"Epoch {epoch}, Loss: {total_loss.numpy():.4f}")


def translate(sentence):
    seq = eng_tokenizer.texts_to_sequences([sentence])
    seq = keras.preprocessing.sequence.pad_sequences(seq,
maxlen=eng_max_len, padding='post')

    enc_output, enc_h, enc_c = encoder(seq)

    dec_input = tf.expand_dims([spa_tokenizer.word_index['<start>']],
0)
    dec_hidden, dec_cell = enc_h, enc_c

    result = ""

    for t in range(spa_max_len):
        predictions, dec_hidden, dec_cell = decoder(
            dec_input, dec_hidden, dec_cell, enc_output
        )
        predicted_id = tf.argmax(predictions[0][0]).numpy()

        if spa_tokenizer.index_word.get(predicted_id) == "<end>":
            break

        result += spa_tokenizer.index_word.get(predicted_id, '') + " "
        dec_input = tf.expand_dims([predicted_id], 0)

    return result.strip()

print("\n===== Translation Results =====")
print("hello →", translate("hello"))
print("thank you →", translate("thank you"))
print("good morning →", translate("good morning"))
```

```
English Vocabulary Size: 11
Spanish Vocabulary Size: 11
Epoch 0, Loss: 7.1949
Epoch 50, Loss: 2.6380
Epoch 100, Loss: 0.0593
Epoch 150, Loss: 0.0189
Epoch 200, Loss: 0.0102
Epoch 250, Loss: 0.0066

===== Translation Results =====
hello → hola
thank you → gracias
good morning → buenos dias
```

Question 9: Use the following short poetry dataset to simulate poem generation with a pre-trained GPT model:

["Roses are red, violets are blue,", "Sugar is sweet, and so are you.", "The moon glows bright in silent skies,", "A bird sings where the soft wind sighs."]

Using this dataset as a reference for poetic structure and language, generate a new 2-4 line poem using a pre-trained GPT model (such as GPT-2). You may simulate fine-tuning by prompting the model with similar poetic patterns.

Include your code, the prompt used, and the generated poem in your answer.

```python
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = """
Roses are red, violets are blue,
Sugar is sweet, and so are you.
The moon glows bright in silent skies,
"""

poem = generator(prompt, max_length=80, num_return_sequences=1)
print(poem[0]['generated_text'])
```

{"model_id":"500c45f3bfd14a9288222832c6d9c1fb","version_major":2,"version_minor":0}

```
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                    | Status     |  |
-----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end
generation.
Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been
set. `max_new_tokens` will take precedence. Please refer to the
documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)


Roses are red, violets are blue,
Sugar is sweet, and so are you.
The moon glows bright in silent skies,
And it's a bright day to see my friends.
The moon glows bright in silent skies.
And it's a bright day to see my friends.
Boom! I see a man! A man-shaped creature.
You look to me, I see a man.
Boom! I see a man.
Boom! I see a man.
Boom! I see a man.
Boom! I see a man.
Boom! I see a man.
Boom! I see a man.
The moon glows bright in silent skies, and it's a bright day to see my
friends.
The moon glows bright in silent skies, and it's a bright day to see my
friends.
Boom! I see a man! A man.
Boom! I see a man.
The moon glows bright in silent skies, and it's a bright day to see my
friends.
The moon glows bright in silent skies, and it's a bright day to see my
friends.
Boom! I see a man! A man.
Boom! I see a man.
Boom! I see a man.
The moon glows bright in silent
```

Question 10: Imagine you are building a creative writing assistant for a publishing

company. The assistant should generate story plots and character descriptions using Generative AI. Describe how you would design the system, including model selection, training data, bias mitigation, and evaluation methods. Explain the real-world challenges you might face.

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompt_plot = """
Generate a fantasy story plot about a young woman who discovers she
can control time.
Include conflict and a moral dilemma.
"""

plot = generator(prompt_plot, max_length=200, num_return_sequences=1)
print("Generated Story Plot:\n")
print(plot[0]['generated_text'])
prompt_character = """
Create a detailed character description for a mysterious detective in
a cyberpunk city.
Include personality traits and backstory.
"""

character = generator(prompt_character, max_length=200,
num_return_sequences=1)
print("\nGenerated Character Description:\n")
print(character[0]['generated_text'])
```

{"model_id":"01705279b20f4b358a2bce7ba1ff1ed6","version_major":2,"version_minor":0}

```
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                   | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end
generation.
Both `max_new_tokens` (=256) and `max_length`(=200) seem to have been
set. `max_new_tokens` will take precedence. Please refer to the
documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_ge
neration)
Setting `pad_token_id` to `eos_token_id`:50256 for open-end
generation.
```

```
Both `max_new_tokens` (=256) and `max_length`(=200) seem to have been
set. `max_new_tokens` will take precedence. Please refer to the
documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_ge
neration)

Generated Story Plot:


Generate a fantasy story plot about a young woman who discovers she
can control time.
Include conflict and a moral dilemma.
Use a variety of non-fiction, fiction, and non-fiction sources to
explore the themes and characters of your story.
Use characters and other fictional characters to create the reader's
own fantasy worlds.
The following works are used as inspiration:
The New York Times bestseller and a top selling novel by the author of
the famous 1984 novel The Great Book of the World. The new book, A
World of Illusion by Michael S. D'Arcy, is a new series of essays and
illustrated short stories by Michael S. D'Arcy and Paul Weyrich. The
original book, The Last Stand of the Great Gatsby by Arthur C. Clarke,
is a new series of essays and illustrated short stories by David H.
Weyrich.
The New York Times bestseller and a top selling novel by the author of
the famous 1984 novel The Great Book of the World. The new book, The
Last Stand of the Great Gatsby by Arthur C. Clarke, is a new series of
essays and illustrated short stories by David H. Weyrich. The original
book, The Last Stand of the Great Gatsby by Arthur C. Clarke, is a new
series of essays and illustrated short stories by David H. Weyrich.

Generated Character Description:


Create a detailed character description for a mysterious detective in
a cyberpunk city.
Include personality traits and backstory.
This section is about the character, not the game. You can help WotC
Wiki by expanding it.

"I am a man of the people, who is all things to all people. I am an
outsider, a man of words, and a man of the people. I am not a man of
words, but a man of the people." —Dane the Engineer [src]

The Dune II is a space-opera-themed space-age game that is about the
Dune's history, culture, and history as a race of humans who work
together to achieve something unique, and for that special reason, a
game about "the people."

Contents show]
```

Gameplay

The game is divided into two parts: one is a survival mode where players pick one of two paths, the first one, to complete the mission. The second part is a story mode where players may choose to take on the role of the Dune II, and the main character of this game, the protagonist.

The story mode is similar to other 3D space-age games (see "Story Mode") where the player character's story is told in the form of a story arc. Characters can be described as being "good